

hurdles. This unreliability makes it difficult to deploy them in critical or complex applications where consistent performance is essential.

Why: The Exception Handling and Recovery pattern provides a standardized solution for building robust and resilient AI agents. It equips them with the agentic capability to anticipate, manage, and recover from operational failures. The pattern involves proactive error detection, such as monitoring tool outputs and API responses, and reactive handling strategies like logging for diagnostics, retrying transient failures, or using fallback mechanisms. For more severe issues, it defines recovery protocols, including reverting to a stable state, self-correction by adjusting its plan, or escalating the problem to a human operator. This systematic approach ensures agents can maintain operational integrity, learn from failures, and function dependably in unpredictable settings.

Rule of thumb: Use this pattern for any AI agent deployed in a dynamic, real-world environment where system failures, tool errors, network issues, or unpredictable inputs are possible and operational reliability is a key requirement.

Visual summary

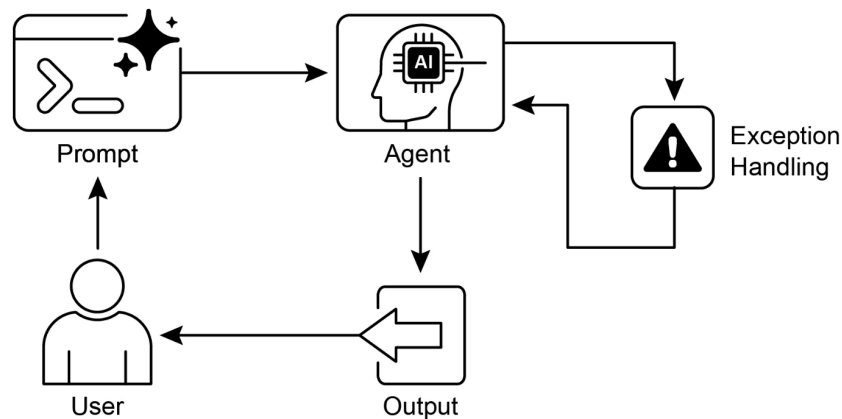


Fig.2: Exception handling pattern

Key Takeaways

Essential points to remember:

- Exception Handling and Recovery is essential for building robust and reliable Agents.
- This pattern involves detecting errors, handling them gracefully, and implementing strategies to recover.
- Error detection can involve validating tool outputs, checking API error codes, and using timeouts.
- Handling strategies include logging, retries, fallbacks, graceful degradation, and notifications.
- Recovery focuses on restoring stable operation through diagnosis, self-correction, or escalation.
- This pattern ensures agents can operate effectively even in unpredictable real-world environments.

Conclusion

This chapter explores the Exception Handling and Recovery pattern, which is essential for developing robust and dependable AI agents. This pattern addresses how AI agents can identify and manage unexpected issues, implement appropriate responses, and recover to a stable operational state. The chapter discusses various aspects of this pattern, including the detection of errors, the handling of these errors through mechanisms such as logging, retries, and fallbacks, and the strategies used to restore the agent or system to proper function. Practical applications of the Exception Handling and Recovery pattern are illustrated across several domains to demonstrate its relevance in handling real-world complexities and potential failures. These applications show how equipping AI agents with exception handling capabilities contributes to their reliability and adaptability in dynamic environments.

References

1. McConnell, S. (2004). *Code Complete (2nd ed.)*. Microsoft Press.
2. Shi, Y., Pei, H., Feng, L., Zhang, Y., & Yao, D. (2024). *Towards Fault Tolerance in Multi-Agent Reinforcement Learning*. arXiv preprint arXiv:2412.00534.
3. O'Neill, V. (2022). *Improving Fault Tolerance and Reliability of Heterogeneous Multi-Agent IoT Systems Using Intelligence Transfer*. *Electronics*, 11(17), 2724.

Chapter 13: Human-in-the-Loop

The Human-in-the-Loop (HITL) pattern represents a pivotal strategy in the development and deployment of Agents. It deliberately interweaves the unique strengths of human cognition—such as judgment, creativity, and nuanced understanding—with the computational power and efficiency of AI. This strategic integration is not merely an option but often a necessity, especially as AI systems become increasingly embedded in critical decision-making processes.

The core principle of HITL is to ensure that AI operates within ethical boundaries, adheres to safety protocols, and achieves its objectives with optimal effectiveness. These concerns are particularly acute in domains characterized by complexity, ambiguity, or significant risk, where the implications of AI errors or misinterpretations can be substantial. In such scenarios, full autonomy—where AI systems function independently without any human intervention—may prove to be imprudent. HITL acknowledges this reality and emphasizes that even with rapidly advancing AI technologies, human oversight, strategic input, and collaborative interactions remain indispensable.

The HITL approach fundamentally revolves around the idea of synergy between artificial and human intelligence. Rather than viewing AI as a replacement for human workers, HITL positions AI as a tool that augments and enhances human capabilities. This augmentation can take various forms, from automating routine tasks to providing data-driven insights that inform human decisions. The end goal is to create a collaborative ecosystem where both humans and AI Agents can leverage their distinct strengths to achieve outcomes that neither could accomplish alone.

In practice, HITL can be implemented in diverse ways. One common approach involves humans acting as validators or reviewers, examining AI outputs to ensure accuracy and identify potential errors. Another implementation involves humans actively guiding AI behavior, providing feedback or making corrections in real-time. In more complex setups, humans may collaborate with AI as partners, jointly solving problems or making decisions through interactive dialog or shared interfaces. Regardless of the specific implementation, the HITL pattern underscores the importance of maintaining human control and oversight, ensuring that AI systems remain aligned with human ethics, values, goals, and societal expectations.

Human-in-the-Loop Pattern Overview

The Human-in-the-Loop (HITL) pattern integrates artificial intelligence with human input to enhance Agent capabilities. This approach acknowledges that optimal AI performance frequently requires a combination of automated processing and human insight, especially in scenarios with high complexity or ethical considerations. Rather than replacing human input, HITL aims to augment human abilities by ensuring that critical judgments and decisions are informed by human understanding.

HITL encompasses several key aspects: Human Oversight, which involves monitoring AI agent performance and output (e.g., via log reviews or real-time dashboards) to ensure adherence to guidelines and prevent undesirable outcomes. Intervention and Correction occurs when an AI agent encounters errors or ambiguous scenarios and may request human intervention; human operators can rectify errors, supply missing data, or guide the agent, which also informs future agent improvements. Human Feedback for Learning is collected and used to refine AI models, prominently in methodologies like reinforcement learning with human feedback, where human preferences directly influence the agent's learning trajectory. Decision Augmentation is where an AI agent provides analyses and recommendations to a human, who then makes the final decision, enhancing human decision-making through AI-generated insights rather than full autonomy. Human-Agent Collaboration is a cooperative interaction where humans and AI agents contribute their respective strengths; routine data processing may be handled by the agent, while creative problem-solving or complex negotiations are managed by the human. Finally, Escalation Policies are established protocols that dictate when and how an agent should escalate tasks to human operators, preventing errors in situations beyond the agent's capability.

Implementing HITL patterns enables the use of Agents in sensitive sectors where full autonomy is not feasible or permitted. It also provides a mechanism for ongoing improvement through feedback loops. For example, in finance, the final approval of a large corporate loan requires a human loan officer to assess qualitative factors like leadership character. Similarly, in the legal field, core principles of justice and accountability demand that a human judge retain final authority over critical decisions like sentencing, which involve complex moral reasoning.

Caveats: Despite its benefits, the HITL pattern has significant caveats, chief among them being a lack of scalability. While human oversight provides high accuracy, operators cannot manage millions of tasks, creating a fundamental trade-off that often requires a hybrid approach combining automation for scale and HITL for

accuracy. Furthermore, the effectiveness of this pattern is heavily dependent on the expertise of the human operators; for example, while an AI can generate software code, only a skilled developer can accurately identify subtle errors and provide the correct guidance to fix them. This need for expertise also applies when using HITL to generate training data, as human annotators may require special training to learn how to correct an AI in a way that produces high-quality data. Lastly, implementing HITL raises significant privacy concerns, as sensitive information must often be rigorously anonymized before it can be exposed to a human operator, adding another layer of process complexity.

Practical Applications & Use Cases

The Human-in-the-Loop pattern is vital across a wide range of industries and applications, particularly where accuracy, safety, ethics, or nuanced understanding are paramount.

- **Content Moderation:** AI agents can rapidly filter vast amounts of online content for violations (e.g., hate speech, spam). However, ambiguous cases or borderline content are escalated to human moderators for review and final decision, ensuring nuanced judgment and adherence to complex policies.
- **Autonomous Driving:** While self-driving cars handle most driving tasks autonomously, they are designed to hand over control to a human driver in complex, unpredictable, or dangerous situations that the AI cannot confidently navigate (e.g., extreme weather, unusual road conditions).
- **Financial Fraud Detection:** AI systems can flag suspicious transactions based on patterns. However, high-risk or ambiguous alerts are often sent to human analysts who investigate further, contact customers, and make the final determination on whether a transaction is fraudulent.
- **Legal Document Review:** AI can quickly scan and categorize thousands of legal documents to identify relevant clauses or evidence. Human legal professionals then review the AI's findings for accuracy, context, and legal implications, especially for critical cases.
- **Customer Support (Complex Queries):** A chatbot might handle routine customer inquiries. If the user's problem is too complex, emotionally charged, or requires empathy that the AI cannot provide, the conversation is seamlessly handed over to a human support agent.
- **Data Labeling and Annotation:** AI models often require large datasets of labeled data for training. Humans are put in the loop to accurately label images, text, or

audio, providing the ground truth that the AI learns from. This is a continuous process as models evolve.

- **Generative AI Refinement:** When an LLM generates creative content (e.g., marketing copy, design ideas), human editors or designers review and refine the output, ensuring it meets brand guidelines, resonates with the target audience, and maintains quality.
- **Autonomous Networks:** AI systems are capable of analyzing alerts and forecasting network issues and traffic anomalies by leveraging key performance indicators (KPIs) and identified patterns. Nevertheless, crucial decisions—such as addressing high-risk alerts—are frequently escalated to human analysts. These analysts conduct further investigation and make the ultimate determination regarding the approval of network changes.

This pattern exemplifies a practical method for AI implementation. It harnesses AI for enhanced scalability and efficiency, while maintaining human oversight to ensure quality, safety, and ethical compliance.

"Human-on-the-loop" is a variation of this pattern where human experts define the overarching policy, and the AI then handles immediate actions to ensure compliance. Let's consider two examples:

- **Automated financial trading system:** In this scenario, a human financial expert sets the overarching investment strategy and rules. For instance, the human might define the policy as: "Maintain a portfolio of 70% tech stocks and 30% bonds, do not invest more than 5% in any single company, and automatically sell any stock that falls 10% below its purchase price." The AI then monitors the stock market in real-time, executing trades instantly when these predefined conditions are met. The AI is handling the immediate, high-speed actions based on the slower, more strategic policy set by the human operator.
- **Modern call center:** In this setup, a human manager establishes high-level policies for customer interactions. For instance, the manager might set rules such as "any call mentioning 'service outage' should be immediately routed to a technical support specialist," or "if a customer's tone of voice indicates high frustration, the system should offer to connect them directly to a human agent." The AI system then handles the initial customer interactions, listening to and interpreting their needs in real-time. It autonomously executes the manager's policies by instantly routing the calls or offering escalations without needing human intervention for each individual case. This allows the AI to manage the high

volume of immediate actions according to the slower, strategic guidance provided by the human operator.

Hands-On Code Example

To demonstrate the Human-in-the-Loop pattern, an ADK agent can identify scenarios requiring human review and initiate an escalation process . This allows for human intervention in situations where the agent's autonomous decision-making capabilities are limited or when complex judgments are required. This is not an isolated feature; other popular frameworks have adopted similar capabilities. LangChain, for instance, also provides tools to implement these types of interactions.

```
from google.adk.agents import Agent
from google.adk.tools.tool_context import ToolContext
from google.adk.callbacks import CallbackContext
from google.adk.models.llm import LlmRequest
from google.genai import types
from typing import Optional

# Placeholder for tools (replace with actual implementations if
needed)
def troubleshoot_issue(issue: str) -> dict:
    return {"status": "success", "report": f"Troubleshooting steps for
{issue}."}

def create_ticket(issue_type: str, details: str) -> dict:
    return {"status": "success", "ticket_id": "TICKET123"}

def escalate_to_human(issue_type: str) -> dict:
    # This would typically transfer to a human queue in a real system
    return {"status": "success", "message": f"Escalated {issue_type}
to a human specialist."}

technical_support_agent = Agent(
    name="technical_support_specialist",
    model="gemini-2.0-flash-exp",
    instruction="""
You are a technical support specialist for our electronics company.
FIRST, check if the user has a support history in
state["customer_info"]["support_history"]. If they do, reference this
history in your responses.
For technical issues:
1. Use the troubleshoot_issue tool to analyze the problem.
2. Guide the user through basic troubleshooting steps.
3. If the issue persists, use create_ticket to log the issue.
```



```

For complex issues beyond basic troubleshooting:
1. Use escalate_to_human to transfer to a human specialist.
Maintain a professional but empathetic tone. Acknowledge the
frustration technical issues can cause, while providing clear steps
toward resolution.
"""
    tools=[troubleshoot_issue, create_ticket, escalate_to_human]
)

def personalization_callback(
    callback_context: CallbackContext, llm_request: LlmRequest
) -> Optional[LlmRequest]:
    """Adds personalization information to the LLM request."""
    # Get customer info from state
    customer_info = callback_context.state.get("customer_info")
    if customer_info:
        customer_name = customer_info.get("name", "valued customer")
        customer_tier = customer_info.get("tier", "standard")
        recent_purchases = customer_info.get("recent_purchases", [])

        personalization_note = (
            f"\nIMPORTANT PERSONALIZATION:\n"
            f"Customer Name: {customer_name}\n"
            f"Customer Tier: {customer_tier}\n"
        )
        if recent_purchases:
            personalization_note += f"Recent Purchases: {'',
'.join(recent_purchases)}\n"

        if llm_request.contents:
            # Add as a system message before the first content
            system_content = types.Content(
                role="system",
            parts=[types.Part(text=personalization_note)]
            )
            llm_request.contents.insert(0, system_content)
    return None # Return None to continue with the modified request

```

This code offers a blueprint for creating a technical support agent using Google's ADK, designed around a HITL framework. The agent acts as an intelligent first line of support, configured with specific instructions and equipped with tools like `troubleshoot_issue`, `create_ticket`, and `escalate_to_human` to manage a complete

support workflow. The escalation tool is a core part of the HITL design, ensuring complex or sensitive cases are passed to human specialists.

A key feature of this architecture is its capacity for deep personalization, achieved through a dedicated callback function. Before contacting the LLM, this function dynamically retrieves customer-specific data—such as their name, tier, and purchase history—from the agent's state. This context is then injected into the prompt as a system message, enabling the agent to provide highly tailored and informed responses that reference the user's history. By combining a structured workflow with essential human oversight and dynamic personalization, this code serves as a practical example of how the ADK facilitates the development of sophisticated and robust AI support solutions.

At Glance

What: AI systems, including advanced LLMs, often struggle with tasks that require nuanced judgment, ethical reasoning, or a deep understanding of complex, ambiguous contexts. Deploying fully autonomous AI in high-stakes environments carries significant risks, as errors can lead to severe safety, financial, or ethical consequences. These systems lack the inherent creativity and common-sense reasoning that humans possess. Consequently, relying solely on automation in critical decision-making processes is often imprudent and can undermine the system's overall effectiveness and trustworthiness.

Why: The Human-in-the-Loop (HITL) pattern provides a standardized solution by strategically integrating human oversight into AI workflows. This agentic approach creates a symbiotic partnership where AI handles computational heavy-lifting and data processing, while humans provide critical validation, feedback, and intervention. By doing so, HITL ensures that AI actions align with human values and safety protocols. This collaborative framework not only mitigates the risks of full automation but also enhances the system's capabilities through continuous learning from human input. Ultimately, this leads to more robust, accurate, and ethical outcomes that neither human nor AI could achieve alone.

Rule of thumb: Use this pattern when deploying AI in domains where errors have significant safety, ethical, or financial consequences, such as in healthcare, finance, or autonomous systems. It is essential for tasks involving ambiguity and nuance that LLMs cannot reliably handle, like content moderation or complex customer support escalations. Employ HITL when the goal is to continuously improve an AI model with

high-quality, human-labeled data or to refine generative AI outputs to meet specific quality standards.

Visual summary:

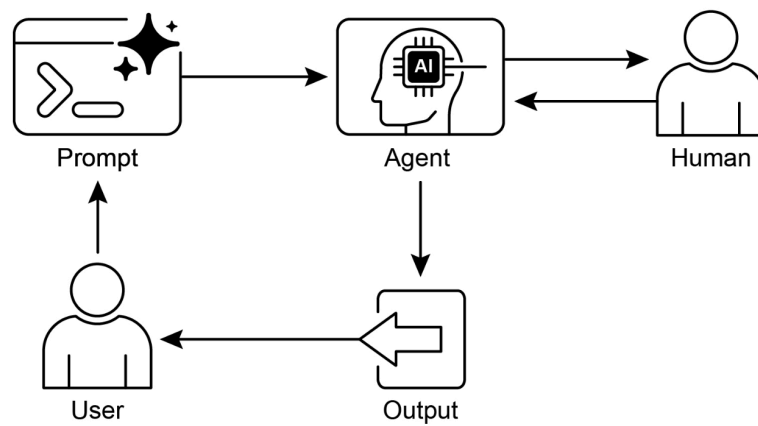


Fig.1: Human in the loop design pattern

Key Takeaways

Key takeaways include:

- Human-in-the-Loop (HITL) integrates human intelligence and judgment into AI workflows.
- It's crucial for safety, ethics, and effectiveness in complex or high-stakes scenarios.
- Key aspects include human oversight, intervention, feedback for learning, and decision augmentation.
- Escalation policies are essential for agents to know when to hand off to a human.
- HITL allows for responsible AI deployment and continuous improvement.

- The primary drawbacks of Human-in-the-Loop are its inherent lack of scalability, creating a trade-off between accuracy and volume, and its dependence on highly skilled domain experts for effective intervention.
- Its implementation presents operational challenges, including the need to train human operators for data generation and to address privacy concerns by anonymizing sensitive information.

Conclusion

This chapter explored the vital Human-in-the-Loop (HITL) pattern, emphasizing its role in creating robust, safe, and ethical AI systems. We discussed how integrating human oversight, intervention, and feedback into agent workflows can significantly enhance their performance and trustworthiness, especially in complex and sensitive domains. The practical applications demonstrated HITL's widespread utility, from content moderation and medical diagnosis to autonomous driving and customer support. The conceptual code example provided a glimpse into how ADK can facilitate these human-agent interactions through escalation mechanisms. As AI capabilities continue to advance, HITL remains a cornerstone for responsible AI development, ensuring that human values and expertise remain central to intelligent system design.

References

1. A Survey of Human-in-the-loop for Machine Learning, Xingjiao Wu, Luwei Xiao, Yixuan Sun, Junhang Zhang, Tianlong Ma, Liang He, <https://arxiv.org/abs/2108.00941>

Chapter 14: Knowledge Retrieval (RAG)

LLMs exhibit substantial capabilities in generating human-like text. However, their knowledge base is typically confined to the data on which they were trained, limiting their access to real-time information, specific company data, or highly specialized details. Knowledge Retrieval (RAG, or Retrieval Augmented Generation), addresses this limitation. RAG enables LLMs to access and integrate external, current, and context-specific information, thereby enhancing the accuracy, relevance, and factual basis of their outputs.

For AI agents, this is crucial as it allows them to ground their actions and responses in real-time, verifiable data beyond their static training. This capability enables them to perform complex tasks accurately, such as accessing the latest company policies to answer a specific question or checking current inventory before placing an order. By integrating external knowledge, RAG transforms agents from simple conversationalists into effective, data-driven tools capable of executing meaningful work.

Knowledge Retrieval (RAG) Pattern Overview

The Knowledge Retrieval (RAG) pattern significantly enhances the capabilities of LLMs by granting them access to external knowledge bases before generating a response. Instead of relying solely on their internal, pre-trained knowledge, RAG allows LLMs to "look up" information, much like a human might consult a book or search the internet. This process empowers LLMs to provide more accurate, up-to-date, and verifiable answers.

When a user poses a question or gives a prompt to an AI system using RAG, the query isn't sent directly to the LLM. Instead, the system first scours a vast external knowledge base—a highly organized library of documents, databases, or web pages—for relevant information. This search is not a simple keyword match; it's a "semantic search" that understands the user's intent and the meaning behind their words. This initial search pulls out the most pertinent snippets or "chunks" of information. These extracted pieces are then "augmented," or added, to the original prompt, creating a richer, more informed query. Finally, this enhanced prompt is sent to the LLM. With this additional context, the LLM can generate a response that is not only fluent and natural but also factually grounded in the retrieved data.

The RAG framework provides several significant benefits. It allows LLMs to access up-to-date information, thereby overcoming the constraints of their static training

data. This approach also reduces the risk of "hallucination"—the generation of false information—by grounding responses in verifiable data. Moreover, LLMs can utilize specialized knowledge found in internal company documents or wikis. A vital advantage of this process is the capability to offer "citations," which pinpoint the exact source of information, thereby enhancing the trustworthiness and verifiability of the AI's responses..

To fully appreciate how RAG functions, it's essential to understand a few core concepts (see Fig.1):

Embeddings: In the context of LLMs, embeddings are numerical representations of text, such as words, phrases, or entire documents. These representations are in the form of a vector, which is a list of numbers. The key idea is to capture the semantic meaning and the relationships between different pieces of text in a mathematical space. Words or phrases with similar meanings will have embeddings that are closer to each other in this vector space. For instance, imagine a simple 2D graph. The word "cat" might be represented by the coordinates (2, 3), while "kitten" would be very close at (2.1, 3.1). In contrast, the word "car" would have a distant coordinate like (8, 1), reflecting its different meaning. In reality, these embeddings are in a much higher-dimensional space with hundreds or even thousands of dimensions, allowing for a very nuanced understanding of language.

Text Similarity: Text similarity refers to the measure of how alike two pieces of text are. This can be at a surface level, looking at the overlap of words (lexical similarity), or at a deeper, meaning-based level. In the context of RAG, text similarity is crucial for finding the most relevant information in the knowledge base that corresponds to a user's query. For instance, consider the sentences: "What is the capital of France?" and "Which city is the capital of France?". While the wording is different, they are asking the same question. A good text similarity model would recognize this and assign a high similarity score to these two sentences, even though they only share a few words. This is often calculated using the embeddings of the texts.

Semantic Similarity and Distance: Semantic similarity is a more advanced form of text similarity that focuses purely on the meaning and context of the text, rather than just the words used. It aims to understand if two pieces of text convey the same concept or idea. Semantic distance is the inverse of this; a high semantic similarity implies a low semantic distance, and vice versa. In RAG, semantic search relies on finding documents with the smallest semantic distance to the user's query. For instance, the phrases "a furry feline companion" and "a domestic cat" have no words in common besides "a". However, a model that understands semantic similarity would

recognize that they refer to the same thing and would consider them to be highly similar. This is because their embeddings would be very close in the vector space, indicating a small semantic distance. This is the "smart search" that allows RAG to find relevant information even when the user's wording doesn't exactly match the text in the knowledge base.

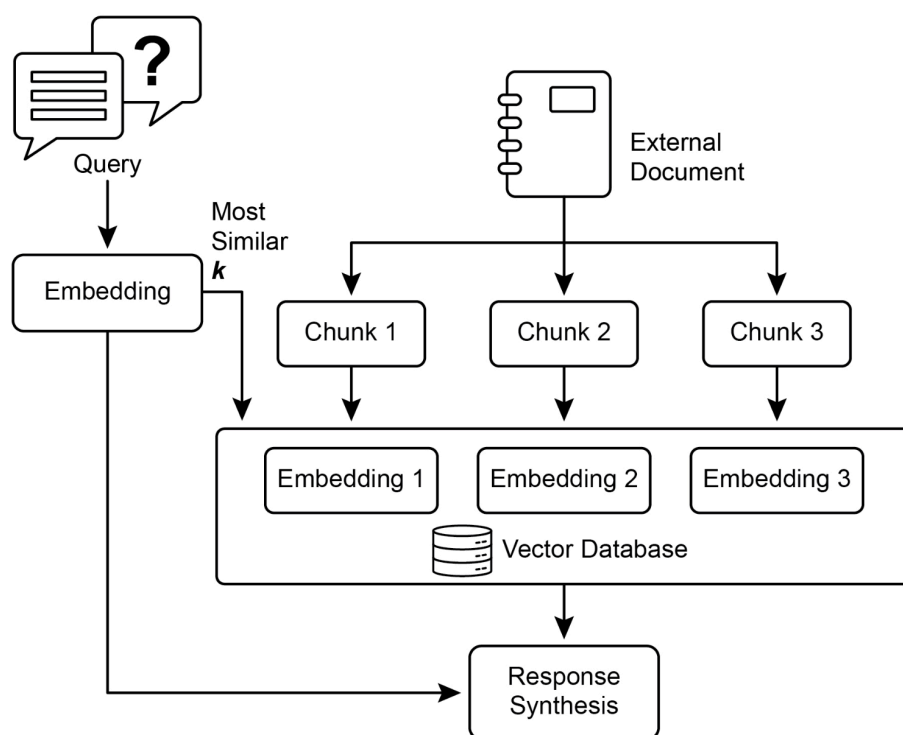


Fig.1: RAG Core Concepts: Chunking, Embeddings, and Vector Database

Chunking of Documents: Chunking is the process of breaking down large documents into smaller, more manageable pieces, or "chunks." For a RAG system to work efficiently, it cannot feed entire large documents into the LLM. Instead, it processes these smaller chunks. The way documents are chunked is important for preserving the context and meaning of the information. For instance, instead of treating a 50-page user manual as a single block of text, a chunking strategy might break it down into sections, paragraphs, or even sentences. For instance, a section on "Troubleshooting" would be a separate chunk from the "Installation Guide." When a user asks a question about a specific problem, the RAG system can then retrieve the most relevant troubleshooting chunk, rather than the entire manual. This makes the

retrieval process faster and the information provided to the LLM more focused and relevant to the user's immediate need. Once documents are chunked, the RAG system must employ a retrieval technique to find the most relevant pieces for a given query. The primary method is vector search, which uses embeddings and semantic distance to find chunks that are conceptually similar to the user's question. An older, but still valuable, technique is BM25, a keyword-based algorithm that ranks chunks based on term frequency without understanding semantic meaning. To get the best of both worlds, hybrid search approaches are often used, combining the keyword precision of BM25 with the contextual understanding of semantic search. This fusion allows for more robust and accurate retrieval, capturing both literal matches and conceptual relevance.

Vector databases: A vector database is a specialized type of database designed to store and query embeddings efficiently. After documents are chunked and converted into embeddings, these high-dimensional vectors are stored in a vector database. Traditional retrieval techniques, like keyword-based search, are excellent at finding documents containing exact words from a query but lack a deep understanding of language. They wouldn't recognize that "furry feline companion" means "cat." This is where vector databases excel. They are built specifically for semantic search. By storing text as numerical vectors, they can find results based on conceptual meaning, not just keyword overlap. When a user's query is also converted into a vector, the database uses highly optimized algorithms (like HNSW - Hierarchical Navigable Small World) to rapidly search through millions of vectors and find the ones that are "closest" in meaning. This approach is far superior for RAG because it uncovers relevant context even if the user's phrasing is completely different from the source documents. In essence, while other techniques search for words, vector databases search for meaning. This technology is implemented in various forms, from managed databases like Pinecone and Weaviate to open-source solutions such as Chroma DB, Milvus, and Qdrant. Even existing databases can be augmented with vector search capabilities, as seen with Redis, Elasticsearch, and Postgres (using the pgvector extension). The core retrieval mechanisms are often powered by libraries like Meta AI's FAISS or Google Research's ScaNN, which are fundamental to the efficiency of these systems.

RAG's Challenges: Despite its power, the RAG pattern is not without its challenges. A primary issue arises when the information needed to answer a query is not confined to a single chunk but is spread across multiple parts of a document or even several documents. In such cases, the retriever might fail to gather all the necessary context, leading to an incomplete or inaccurate answer. The system's effectiveness is also

highly dependent on the quality of the chunking and retrieval process; if irrelevant chunks are retrieved, it can introduce noise and confuse the LLM. Furthermore, effectively synthesizing information from potentially contradictory sources remains a significant hurdle for these systems. Besides that, another challenge is that RAG requires the entire knowledge base to be pre-processed and stored in specialized databases, such as vector or graph databases, which is a considerable undertaking. Consequently, this knowledge requires periodic reconciliation to remain up-to-date, a crucial task when dealing with evolving sources like company wikis. This entire process can have a noticeable impact on performance, increasing latency, operational costs, and the number of tokens used in the final prompt.

In summary, the Retrieval-Augmented Generation (RAG) pattern represents a significant leap forward in making AI more knowledgeable and reliable. By seamlessly integrating an external knowledge retrieval step into the generation process, RAG addresses some of the core limitations of standalone LLMs. The foundational concepts of embeddings and semantic similarity, combined with retrieval techniques like keyword and hybrid search, allow the system to intelligently find relevant information, which is made manageable through strategic chunking. This entire retrieval process is powered by specialized vector databases designed to store and efficiently query millions of embeddings at scale. While challenges in retrieving fragmented or contradictory information persist, RAG empowers LLMs to produce answers that are not only contextually appropriate but also anchored in verifiable facts, fostering greater trust and utility in AI.

Graph RAG: GraphRAG is an advanced form of Retrieval-Augmented Generation that utilizes a knowledge graph instead of a simple vector database for information retrieval. It answers complex queries by navigating the explicit relationships (edges) between data entities (nodes) within this structured knowledge base. A key advantage is its ability to synthesize answers from information fragmented across multiple documents, a common failing of traditional RAG. By understanding these connections, GraphRAG provides more contextually accurate and nuanced responses.

Use cases include complex financial analysis, connecting companies to market events, and scientific research for discovering relationships between genes and diseases. The primary drawback, however, is the significant complexity, cost, and expertise required to build and maintain a high-quality knowledge graph. This setup is also less flexible and can introduce higher latency compared to simpler vector search systems. The system's effectiveness is entirely dependent on the quality and completeness of the underlying graph structure. Consequently, GraphRAG offers superior contextual reasoning for intricate questions but at a much higher implementation and

maintenance cost. In summary, it excels where deep, interconnected insights are more critical than the speed and simplicity of standard RAG.

Agentic RAG: An evolution of this pattern, known as **Agentic RAG** (see Fig.2), introduces a reasoning and decision-making layer to significantly enhance the reliability of information extraction. Instead of just retrieving and augmenting, an "agent"—a specialized AI component—acts as a critical gatekeeper and refiner of knowledge. Rather than passively accepting the initially retrieved data, this agent actively interrogates its quality, relevance, and completeness, as illustrated by the following scenarios.

First, an agent excels at reflection and source validation. If a user asks, "What is our company's policy on remote work?" a standard RAG might pull up a 2020 blog post alongside the official 2025 policy document. The agent, however, would analyze the documents' metadata, recognize the 2025 policy as the most current and authoritative source, and discard the outdated blog post before sending the correct context to the LLM for a precise answer.

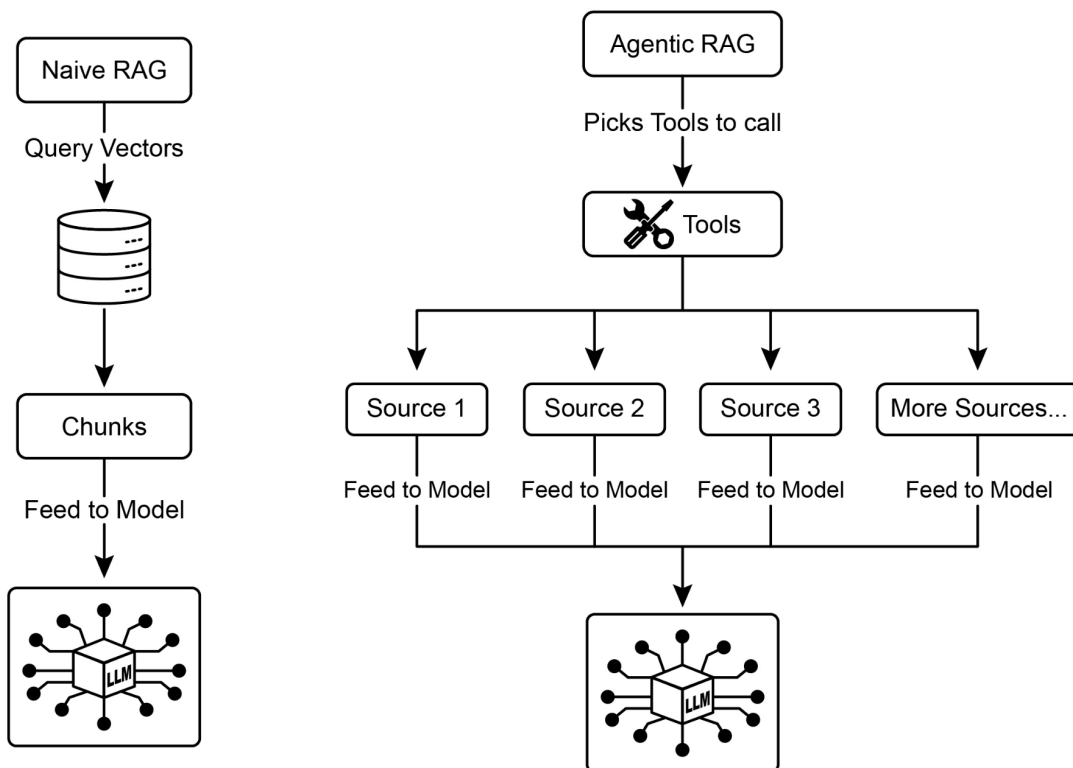


Fig.2: Agentic RAG introduces a reasoning agent that actively evaluates, reconciles, and refines retrieved information to ensure a more accurate and trustworthy final response.

Second, an agent is adept at reconciling knowledge conflicts. Imagine a financial analyst asks, "What was Project Alpha's Q1 budget?" The system retrieves two documents: an initial proposal stating a €50,000 budget and a finalized financial report listing it as €65,000. An Agentic RAG would identify this contradiction, prioritize the financial report as the more reliable source, and provide the LLM with the verified figure, ensuring the final answer is based on the most accurate data.

Third, an agent can perform multi-step reasoning to synthesize complex answers. If a user asks, "How do our product's features and pricing compare to Competitor X's?" the agent would decompose this into separate sub-queries. It would initiate distinct searches for its own product's features, its pricing, Competitor X's features, and Competitor X's pricing. After gathering these individual pieces of information, the agent would synthesize them into a structured, comparative context before feeding it to the LLM, enabling a comprehensive response that a simple retrieval could not have produced.

Fourth, an agent can identify knowledge gaps and use external tools. Suppose a user asks, "What was the market's immediate reaction to our new product launched yesterday?" The agent searches the internal knowledge base, which is updated weekly, and finds no relevant information. Recognizing this gap, it can then activate a tool—such as a live web-search API—to find recent news articles and social media sentiment. The agent then uses this freshly gathered external information to provide an up-to-the-minute answer, overcoming the limitations of its static internal database.

Challenges of Agentic RAG: While powerful, the agentic layer introduces its own set of challenges. The primary drawback is a significant increase in complexity and cost. Designing, implementing, and maintaining the agent's decision-making logic and tool integrations requires substantial engineering effort and adds to computational expenses. This complexity can also lead to increased latency, as the agent's cycles of reflection, tool use, and multi-step reasoning take more time than a standard, direct retrieval process. Furthermore, the agent itself can become a new source of error; a flawed reasoning process could cause it to get stuck in useless loops, misinterpret a task, or improperly discard relevant information, ultimately degrading the quality of the final response.

In summary: Agentic RAG represents a sophisticated evolution of the standard retrieval pattern, transforming it from a passive data pipeline into an active, problem-solving framework. By embedding a reasoning layer that can evaluate sources, reconcile conflicts, decompose complex questions, and use external tools, agents dramatically improve the reliability and depth of the generated answers. This advancement makes the AI more trustworthy and capable, though it comes with important trade-offs in system complexity, latency, and cost that must be carefully managed.

Practical Applications & Use Cases

Knowledge Retrieval (RAG) is changing how Large Language Models (LLMs) are utilized across various industries, enhancing their ability to provide more accurate and contextually relevant responses.

Applications include:

- **Enterprise Search and Q&A:** Organizations can develop internal chatbots that respond to employee inquiries using internal documentation such as HR policies, technical manuals, and product specifications. The RAG system extracts relevant sections from these documents to inform the LLM's response.
- **Customer Support and Helpdesks:** RAG-based systems can offer precise and consistent responses to customer queries by accessing information from product manuals, frequently asked questions (FAQs), and support tickets. This can reduce the need for direct human intervention for routine issues.
- **Personalized Content Recommendation:** Instead of basic keyword matching, RAG can identify and retrieve content (articles, products) that is semantically related to a user's preferences or previous interactions, leading to more relevant recommendations.
- **News and Current Events Summarization:** LLMs can be integrated with real-time news feeds. When prompted about a current event, the RAG system retrieves recent articles, allowing the LLM to produce an up-to-date summary.

By incorporating external knowledge, RAG extends the capabilities of LLMs beyond simple communication to function as knowledge processing systems.

Hands-On Code Example (ADK)

To illustrate the Knowledge Retrieval (RAG) pattern, let's see three examples.

First, is how to use Google Search to do RAG and ground LLMs to search results. Since RAG involves accessing external information, the Google Search tool is a direct example of a built-in retrieval mechanism that can augment an LLM's knowledge.

```
from google.adk.tools import google_search
from google.adk.agents import Agent

search_agent = Agent(
    name="research_assistant",
    model="gemini-2.0-flash-exp",
    instruction="You help users research topics. When asked, use the Google Search tool",
    tools=[google_search]
)
```

Second, this section explains how to utilize Vertex AI RAG capabilities within the Google ADK. The code provided demonstrates the initialization of VertexAiRagMemoryService from the ADK. This allows for establishing a connection to a Google Cloud Vertex AI RAG Corpus. The service is configured by specifying the corpus resource name and optional parameters such as SIMILARITY_TOP_K and VECTOR_DISTANCE_THRESHOLD. These parameters influence the retrieval process. SIMILARITY_TOP_K defines the number of top similar results to be retrieved. VECTOR_DISTANCE_THRESHOLD sets a limit on the semantic distance for the retrieved results. This setup enables agents to perform scalable and persistent semantic knowledge retrieval from the designated RAG Corpus. The process effectively integrates Google Cloud's RAG functionalities into an ADK agent, thereby supporting the development of responses grounded in factual data.

```
# Import the necessary VertexAiRagMemoryService class from the
google.adk.memory module.
from google.adk.memory import VertexAiRagMemoryService

RAG_CORPUS_RESOURCE_NAME =
"projects/your-gcp-project-id/locations/us-central1/ragCorpora/your-c
orpus-id"

# Define an optional parameter for the number of top similar results
```

```

to retrieve.
# This controls how many relevant document chunks the RAG service
will return.
SIMILARITY_TOP_K = 5

# Define an optional parameter for the vector distance threshold.
# This threshold determines the maximum semantic distance allowed for
retrieved results;
# results with a distance greater than this value might be filtered
out.
VECTOR_DISTANCE_THRESHOLD = 0.7

# Initialize an instance of VertexAiRagMemoryService.
# This sets up the connection to your Vertex AI RAG Corpus.
# - rag_corpus: Specifies the unique identifier for your RAG Corpus.
# - similarity_top_k: Sets the maximum number of similar results to
fetch.
# - vector_distance_threshold: Defines the similarity threshold for
filtering results.
memory_service = VertexAiRagMemoryService(
    rag_corpus=RAG_CORPUS_RESOURCE_NAME,
    similarity_top_k=SIMILARITY_TOP_K,
    vector_distance_threshold=VECTOR_DISTANCE_THRESHOLD
)

```

Hands-On Code Example (LangChain)

Third, let's walk through a complete example using LangChain.

```

import os
import requests
from typing import List, Dict, Any, TypedDict
from langchain_community.document_loaders import TextLoader

from langchain_core.documents import Document
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_community.embeddings import OpenAIEmbeddings
from langchain_community.vectorstores import Weaviate
from langchain_openai import ChatOpenAI
from langchain.text_splitter import CharacterTextSplitter
from langchain.schema.runnable import RunnablePassthrough
from langgraph.graph import StateGraph, END
import weaviate
from weaviate.embedded import EmbeddedOptions

```

```

import dotenv

# Load environment variables (e.g., OPENAI_API_KEY)
dotenv.load_dotenv()
# Set your OpenAI API key (ensure it's loaded from .env or set here)
# os.environ["OPENAI_API_KEY"] = "YOUR_OPENAI_API_KEY"

# --- 1. Data Preparation (Preprocessing) ---
# Load data
url =
"https://github.com/langchain-ai/langchain/blob/master/docs/docs/how_
to/state_of_the_union.txt"
res = requests.get(url)

with open("state_of_the_union.txt", "w") as f:
    f.write(res.text)

loader = TextLoader('./state_of_the_union.txt')
documents = loader.load()

# Chunk documents
text_splitter = CharacterTextSplitter(chunk_size=500,
chunk_overlap=50)
chunks = text_splitter.split_documents(documents)

# Embed and store chunks in Weaviate
client = weaviate.Client(
    embedded_options = EmbeddedOptions()
)

vectorstore = Weaviate.from_documents(
    client = client,
    documents = chunks,
    embedding = OpenAIEmbeddings(),
    by_text = False
)

# Define the retriever
retriever = vectorstore.as_retriever()

# Initialize LLM
llm = ChatOpenAI(model_name="gpt-3.5-turbo", temperature=0)

# --- 2. Define the State for LangGraph ---
class RAGGraphState(TypedDict):
    question: str

```

```

    documents: List[Document]
    generation: str

# --- 3. Define the Nodes (Functions) ---

def retrieve_documents_node(state: RAGGraphState) -> RAGGraphState:
    """Retrieves documents based on the user's question."""
    question = state["question"]
    documents = retriever.invoke(question)
    return {"documents": documents, "question": question,
"generation": ""}

def generate_response_node(state: RAGGraphState) -> RAGGraphState:
    """Generates a response using the LLM based on retrieved
documents."""
    question = state["question"]
    documents = state["documents"]

    # Prompt template from the PDF
    template = """You are an assistant for question-answering tasks.
Use the following pieces of retrieved context to answer the question.
If you don't know the answer, just say that you don't know.
Use three sentences maximum and keep the answer concise.
Question: {question}
Context: {context}
Answer:
"""
    prompt = ChatPromptTemplate.from_template(template)

    # Format the context from the documents
    context = "\n\n".join([doc.page_content for doc in documents])

    # Create the RAG chain
    rag_chain = prompt | llm | StrOutputParser()

    # Invoke the chain
    generation = rag_chain.invoke({"context": context, "question":
question})
    return {"question": question, "documents": documents,
"generation": generation}

# --- 4. Build the LangGraph Graph ---

workflow = StateGraph(RAGGraphState)

# Add nodes
workflow.add_node("retrieve", retrieve_documents_node)

```



```

workflow.add_node("generate", generate_response_node)

# Set the entry point
workflow.set_entry_point("retrieve")

# Add edges (transitions)
workflow.add_edge("retrieve", "generate")
workflow.add_edge("generate", END)

# Compile the graph
app = workflow.compile()

# --- 5. Run the RAG Application ---
if __name__ == "__main__":
    print("\n--- Running RAG Query ---")
    query = "What did the president say about Justice Breyer"
    inputs = {"question": query}
    for s in app.stream(inputs):
        print(s)

    print("\n--- Running another RAG Query ---")
    query_2 = "What did the president say about the economy?"
    inputs_2 = {"question": query_2}
    for s in app.stream(inputs_2):
        print(s)

```

This Python code illustrates a Retrieval-Augmented Generation (RAG) pipeline implemented with LangChain and LangGraph. The process begins with the creation of a knowledge base derived from a text document, which is segmented into chunks and transformed into embeddings. These embeddings are then stored in a Weaviate vector store, facilitating efficient information retrieval. A StateGraph in LangGraph is utilized to manage the workflow between two key functions: `retrieve_documents_node` and `generate_response_node`. The `retrieve_documents_node` function queries the vector store to identify relevant document chunks based on the user's input. Subsequently, the `generate_response_node` function utilizes the retrieved information and a predefined prompt template to produce a response using an OpenAI Large Language Model (LLM). The `app.stream` method allows the execution of queries through the RAG pipeline, demonstrating the system's capacity to generate contextually relevant outputs.

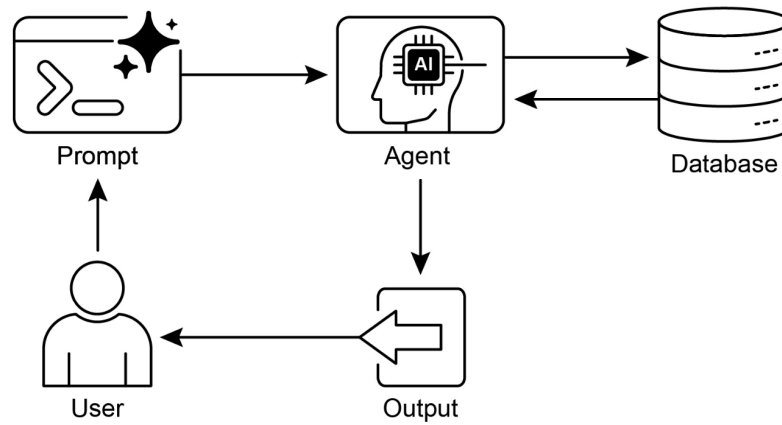
At Glance

What: LLMs possess impressive text generation abilities but are fundamentally limited by their training data. This knowledge is static, meaning it doesn't include real-time information or private, domain-specific data. Consequently, their responses can be outdated, inaccurate, or lack the specific context required for specialized tasks. This gap restricts their reliability for applications demanding current and factual answers.

Why: The Retrieval-Augmented Generation (RAG) pattern provides a standardized solution by connecting LLMs to external knowledge sources. When a query is received, the system first retrieves relevant information snippets from a specified knowledge base. These snippets are then appended to the original prompt, enriching it with timely and specific context. This augmented prompt is then sent to the LLM, enabling it to generate a response that is accurate, verifiable, and grounded in external data. This process effectively transforms the LLM from a closed-book reasoner into an open-book one, significantly enhancing its utility and trustworthiness.

Rule of thumb: Use this pattern when you need an LLM to answer questions or generate content based on specific, up-to-date, or proprietary information that was not part of its original training data. It is ideal for building Q&A systems over internal documents, customer support bots, and applications requiring verifiable, fact-based responses with citations.

Visual summary



Knowledge Retrieval pattern: an AI agent to query and retrieve information from structured databases

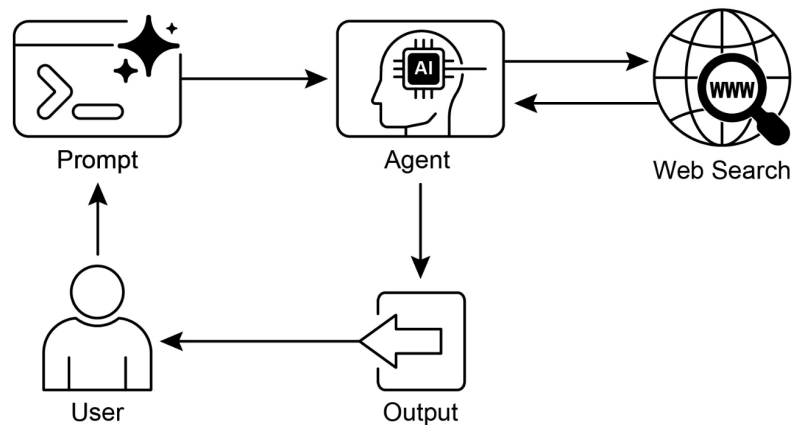


Fig. 3: Knowledge Retrieval pattern: an AI agent to find and synthesize information from the public internet in response to user queries.

Key Takeaways

- Knowledge Retrieval (RAG) enhances LLMs by allowing them to access external, up-to-date, and specific information.
- The process involves Retrieval (searching a knowledge base for relevant snippets) and Augmentation (adding these snippets to the LLM's prompt).
- RAG helps LLMs overcome limitations like outdated training data, reduces "hallucinations," and enables domain-specific knowledge integration.
- RAG allows for attributable answers, as the LLM's response is grounded in retrieved sources.
- GraphRAG leverages a knowledge graph to understand the relationships between different pieces of information, allowing it to answer complex questions that require synthesizing data from multiple sources.

- Agentic RAG moves beyond simple information retrieval by using an intelligent agent to actively reason about, validate, and refine external knowledge, ensuring a more accurate and reliable answer.
- Practical applications span enterprise search, customer support, legal research, and personalized recommendations.

Conclusion

In conclusion, Retrieval-Augmented Generation (RAG) addresses the core limitation of a Large Language Model's static knowledge by connecting it to external, up-to-date data sources. The process works by first retrieving relevant information snippets and then augmenting the user's prompt, enabling the LLM to generate more accurate and contextually aware responses. This is made possible by foundational technologies like embeddings, semantic search, and vector databases, which find information based on meaning rather than just keywords. By grounding outputs in verifiable data, RAG significantly reduces factual errors and allows for the use of proprietary information, enhancing trust through citations.

An advanced evolution, Agentic RAG, introduces a reasoning layer that actively validates, reconciles, and synthesizes retrieved knowledge for even greater reliability. Similarly, specialized approaches like GraphRAG leverage knowledge graphs to navigate explicit data relationships, allowing the system to synthesize answers to highly complex, interconnected queries. This agent can resolve conflicting information, perform multi-step queries, and use external tools to find missing data. While these advanced methods add complexity and latency, they drastically improve the depth and trustworthiness of the final response. Practical applications for these patterns are already transforming industries, from enterprise search and customer support to personalized content delivery. Despite the challenges, RAG is a crucial pattern for making AI more knowledgeable, reliable, and useful. Ultimately, it transforms LLMs from closed-book conversationalists into powerful, open-book reasoning tools.

References

1. Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. <https://arxiv.org/abs/2005.11401>
2. Google AI for Developers Documentation. *Retrieval Augmented Generation -* <https://cloud.google.com/vertex-ai/generative-ai/docs/rag-engine/rag-overview>

3. Retrieval-Augmented Generation with Graphs (GraphRAG),
<https://arxiv.org/abs/2501.00309>
4. LangChain and LangGraph: Leonie Monigatti, "Retrieval-Augmented Generation (RAG): From Theory to LangChain Implementation,"
<https://medium.com/data-science/retrieval-augmented-generation-rag-from-theory-to-langchain-implementation-4e9bd5f6a4f2>
5. Google Cloud Vertex AI RAG Corpus
<https://cloud.google.com/vertex-ai/generative-ai/docs/rag-engine/manage-your-our-rag-corpus#corpus-management>

Chapter 15: Inter-Agent Communication (A2A)

Individual AI agents often face limitations when tackling complex, multifaceted problems, even with advanced capabilities. To overcome this, Inter-Agent Communication (A2A) enables diverse AI agents, potentially built with different frameworks, to collaborate effectively. This collaboration involves seamless coordination, task delegation, and information exchange.

Google's A2A protocol is an open standard designed to facilitate this universal communication. This chapter will explore A2A, its practical applications, and its implementation within the Google ADK.

Inter-Agent Communication Pattern Overview

The Agent2Agent (A2A) protocol is an open standard designed to enable communication and collaboration between different AI agent frameworks. It ensures interoperability, allowing AI agents developed with technologies like LangGraph, CrewAI, or Google ADK to work together regardless of their origin or framework differences.

A2A is supported by a range of technology companies and service providers, including Atlassian, Box, LangChain, MongoDB, Salesforce, SAP, and ServiceNow. Microsoft plans to integrate A2A into Azure AI Foundry and Copilot Studio, demonstrating its commitment to open protocols. Additionally, Auth0 and SAP are integrating A2A support into their platforms and agents.

As an open-source protocol, A2A welcomes community contributions to facilitate its evolution and widespread adoption.

Core Concepts of A2A

The A2A protocol provides a structured approach for agent interactions, built upon several core concepts. A thorough grasp of these concepts is crucial for anyone developing or integrating with A2A-compliant systems. The foundational pillars of A2A include Core Actors, Agent Card, Agent Discovery, Communication and Tasks, Interaction mechanisms, and Security, all of which will be reviewed in detail.

Core Actors: A2A involves three main entities:

- User: Initiates requests for agent assistance.
- A2A Client (Client Agent): An application or AI agent that acts on the user's behalf to request actions or information.
- A2A Server (Remote Agent): An AI agent or system that provides an HTTP endpoint to process client requests and return results. The remote agent operates as an "opaque" system, meaning the client does not need to understand its internal operational details.

Agent Card: An agent's digital identity is defined by its Agent Card, usually a JSON file. This file contains key information for client interaction and automatic discovery, including the agent's identity, endpoint URL, and version. It also details supported capabilities like streaming or push notifications, specific skills, default input/output modes, and authentication requirements. Below is an example of an Agent Card for a WeatherBot.

```
{
  "name": "WeatherBot",
  "description": "Provides accurate weather forecasts and historical data.",
  "url": "http://weather-service.example.com/a2a",
  "version": "1.0.0",
  "capabilities": {
    "streaming": true,
    "pushNotifications": false,
    "stateTransitionHistory": true
  },
  "authentication": {
    "schemes": [
      "apiKey"
    ]
  },
  "defaultInputModes": [
    "text"
  ],
  "defaultOutputModes": [
    "text"
  ],
  "skills": [
    {
      "id": "get_current_weather",
```



```

    "name": "Get Current Weather",
    "description": "Retrieve real-time weather for any location.",
    "inputModes": [
        "text"
    ],
    "outputModes": [
        "text"
    ],
    "examples": [
        "What's the weather in Paris?",
        "Current conditions in Tokyo"
    ],
    "tags": [
        "weather",
        "current",
        "real-time"
    ]
},
{
    "id": "get_forecast",
    "name": "Get Forecast",
    "description": "Get 5-day weather predictions.",
    "inputModes": [
        "text"
    ],
    "outputModes": [
        "text"
    ],
    "examples": [
        "5-day forecast for New York",
        "Will it rain in London this weekend?"
    ],
    "tags": [
        "weather",
        "forecast",
        "prediction"
    ]
}
]
}

```

Agent discovery: it allows clients to find Agent Cards, which describe the capabilities of available A2A Servers. Several strategies exist for this process:

- Well-Known URI: Agents host their Agent Card at a standardized path (e.g.,

/well-known/agent.json). This approach offers broad, often automated, accessibility for public or domain-specific use.

- **Curated Registries:** These provide a centralized catalog where Agent Cards are published and can be queried based on specific criteria. This is well-suited for enterprise environments needing centralized management and access control.
- **Direct Configuration:** Agent Card information is embedded or privately shared. This method is appropriate for closely coupled or private systems where dynamic discovery isn't crucial.

Regardless of the chosen method, it is important to secure Agent Card endpoints. This can be achieved through access control, mutual TLS (mTLS), or network restrictions, especially if the card contains sensitive (though non-secret) information.

Communications and Tasks: In the A2A framework, communication is structured around asynchronous tasks, which represent the fundamental units of work for long-running processes. Each task is assigned a unique identifier and moves through a series of states—such as submitted, working, or completed—a design that supports parallel processing in complex operations. Communication between agents occurs through a Message.

This communication contains attributes, which are key-value metadata describing the message (like its priority or creation time), and one or more parts, which carry the actual content being delivered, such as plain text, files, or structured JSON data. The tangible outputs generated by an agent during a task are called artifacts. Like messages, artifacts are also composed of one or more parts and can be streamed incrementally as results become available. All communication within the A2A framework is conducted over HTTP(S) using the JSON-RPC 2.0 protocol for payloads. To maintain continuity across multiple interactions, a server-generated contextId is used to group related tasks and preserve context.

Interaction Mechanisms: Request/Response (Polling) Server-Sent Events (SSE). A2A provides multiple interaction methods to suit a variety of AI application needs, each with a distinct mechanism:

- **Synchronous Request/Response:** For quick, immediate operations. In this model, the client sends a request and actively waits for the server to process it and return a complete response in a single, synchronous exchange.
- **Asynchronous Polling:** Suited for tasks that take longer to process. The client sends a request, and the server immediately acknowledges it with a "working" status and a task ID. The client is then free to perform other actions and can

periodically poll the server by sending new requests to check the status of the task until it is marked as "completed" or "failed."

- Streaming Updates (Server-Sent Events - SSE): Ideal for receiving real-time, incremental results. This method establishes a persistent, one-way connection from the server to the client. It allows the remote agent to continuously push updates, such as status changes or partial results, without the client needing to make multiple requests.
- Push Notifications (Webhooks): Designed for very long-running or resource-intensive tasks where maintaining a constant connection or frequent polling is inefficient. The client can register a webhook URL, and the server will send an asynchronous notification (a "push") to that URL when the task's status changes significantly (e.g., upon completion).

The Agent Card specifies whether an agent supports streaming or push notification capabilities. Furthermore, A2A is modality-agnostic, meaning it can facilitate these interaction patterns not just for text, but also for other data types like audio and video, enabling rich, multimodal AI applications. Both streaming and push notification capabilities are specified within the Agent Card.

```
#Synchronous Request Example
{
  "jsonrpc": "2.0",
  "id": "1",
  "method": "sendTask",
  "params": {
    "id": "task-001",
    "sessionId": "session-001",
    "message": {
      "role": "user",
      "parts": [
        {
          "type": "text",
          "text": "What is the exchange rate from USD to EUR?"
        }
      ]
    }
  },
  "acceptedOutputModes": ["text/plain"],
  "historyLength": 5
}
```

The synchronous request uses the `sendTask` method, where the client asks for and expects a single, complete answer to its query. In contrast, the streaming request uses the `sendTaskSubscribe` method to establish a persistent connection, allowing the agent to send back multiple, incremental updates or partial results over time.

```
# Streaming Request Example
{
  "jsonrpc": "2.0",
  "id": "2",
  "method": "sendTaskSubscribe",
  "params": {
    "id": "task-002",
    "sessionId": "session-001",
    "message": {
      "role": "user",
      "parts": [
        {
          "type": "text",
          "text": "What's the exchange rate for JPY to GBP today?"
        }
      ]
    },
    "acceptedOutputModes": ["text/plain"],
    "historyLength": 5
  }
}
```

Security: Inter-Agent Communication (A2A): Inter-Agent Communication (A2A) is a vital component of system architecture, enabling secure and seamless data exchange among agents. It ensures robustness and integrity through several built-in mechanisms.

Mutual Transport Layer Security (TLS): Encrypted and authenticated connections are established to prevent unauthorized access and data interception, ensuring secure communication.

Comprehensive Audit Logs: All inter-agent communications are meticulously recorded, detailing information flow, involved agents, and actions. This audit trail is crucial for accountability, troubleshooting, and security analysis.

Agent Card Declaration: Authentication requirements are explicitly declared in the Agent Card, a configuration artifact outlining the agent's identity, capabilities, and security policies. This centralizes and simplifies authentication management.

Credential Handling: Agents typically authenticate using secure credentials like OAuth 2.0 tokens or API keys, passed via HTTP headers. This method prevents credential exposure in URLs or message bodies, enhancing overall security.

A2A vs. MCP

A2A is a protocol that complements Anthropic's Model Context Protocol (MCP) (see Fig. 1). While MCP focuses on structuring context for agents and their interaction with external data and tools, A2A facilitates coordination and communication among agents, enabling task delegation and collaboration.

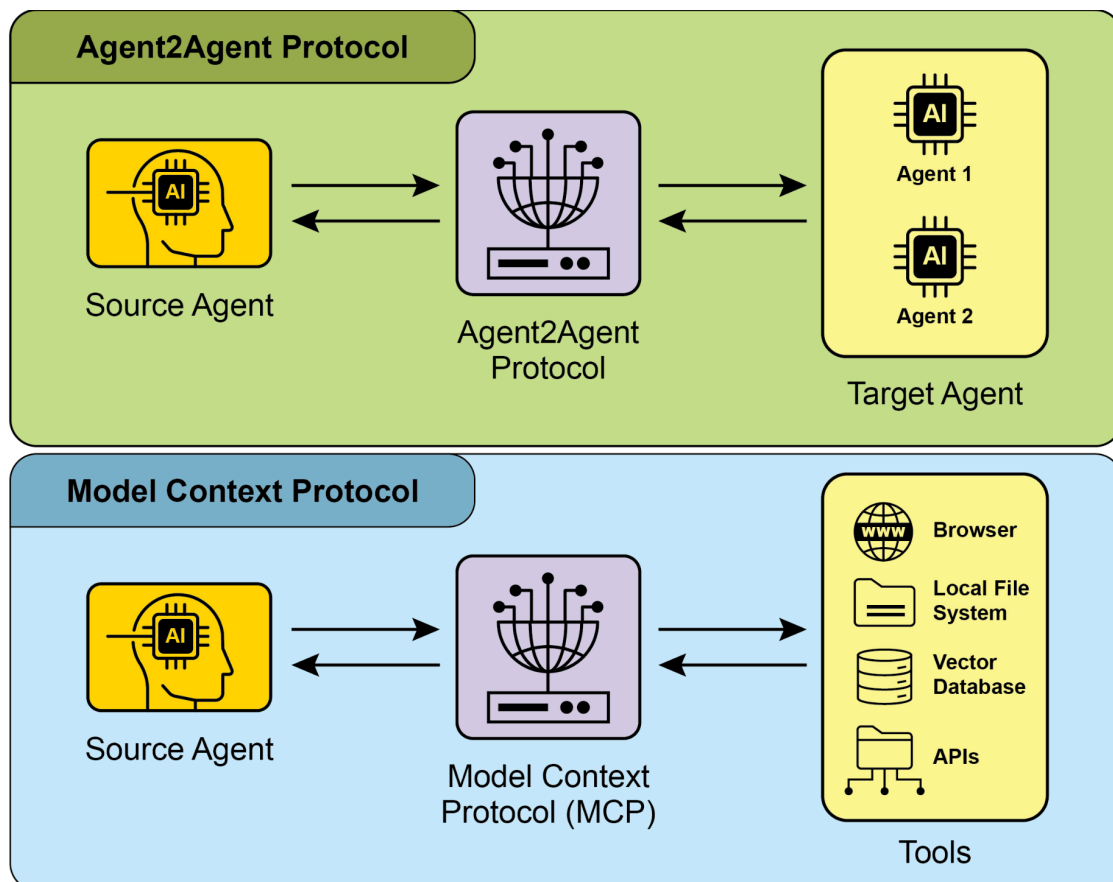


Fig.1: Comparison A2A and MCP Protocols

The goal of A2A is to enhance efficiency, reduce integration costs, and foster innovation and interoperability in the development of complex, multi-agent AI

systems. Therefore, a thorough understanding of A2A's core components and operational methods is essential for its effective design, implementation, and application in building collaborative and interoperable AI agent systems..

Practical Applications & Use Cases

Inter-Agent Communication is indispensable for building sophisticated AI solutions across diverse domains, enabling modularity, scalability, and enhanced intelligence.

- **Multi-Framework Collaboration:** A2A's primary use case is enabling independent AI agents, regardless of their underlying frameworks (e.g., ADK, LangChain, CrewAI), to communicate and collaborate. This is fundamental for building complex multi-agent systems where different agents specialize in different aspects of a problem.
- **Automated Workflow Orchestration:** In enterprise settings, A2A can facilitate complex workflows by enabling agents to delegate and coordinate tasks. For instance, an agent might handle initial data collection, then delegate to another agent for analysis, and finally to a third for report generation, all communicating via the A2A protocol.
- **Dynamic Information Retrieval:** Agents can communicate to retrieve and exchange real-time information. A primary agent might request live market data from a specialized "data fetching agent," which then uses external APIs to gather the information and send it back.

Hands-On Code Example

Let's examine the practical applications of the A2A protocol. The repository at <https://github.com/google-a2a/a2a-samples/tree/main/samples> provides examples in Java, Go, and Python that illustrate how various agent frameworks, such as LangGraph, CrewAI, Azure AI Foundry, and AG2, can communicate using A2A. All code in this repository is released under the Apache 2.0 license. To further illustrate A2A's core concepts, we will review code excerpts focusing on setting up an A2A Server using an ADK-based agent with Google-authenticated tools. Looking at https://github.com/google-a2a/a2a-samples/blob/main/samples/python/agents/birthday_planner_adk/calendar_agent/adk_agent.py

```
import datetime
from google.adk.agents import LlmAgent # type: ignore[import-untyped]
from google.adk.tools.google_api_tool import CalendarToolset # type:
```

```

ignore[import-untyped]

async def create_agent(client_id, client_secret) -> LlmAgent:
    """Constructs the ADK agent."""
    toolset = CalendarToolset(client_id=client_id,
client_secret=client_secret)
    return LlmAgent(
        model='gemini-2.0-flash-001',
        name='calendar_agent',
        description="An agent that can help manage a user's calendar",
        instruction=f"""
You are an agent that can help manage a user's calendar.

Users will request information about the state of their calendar
or to make changes to their calendar. Use the provided tools for
interacting with the calendar API.

If not specified, assume the calendar the user wants is the 'primary'
calendar.

When using the Calendar API tools, use well-formed RFC3339
timestamps.

Today is {datetime.datetime.now()}.
""",
        tools=await toolset.get_tools(),
    )

```

This Python code defines an asynchronous function `create\_agent` that constructs an ADK LlmAgent. It begins by initializing a `CalendarToolset` using the provided client credentials to access the Google Calendar API. Subsequently, an `LlmAgent` instance is created, configured with a specified Gemini model, a descriptive name, and instructions for managing a user's calendar. The agent is furnished with calendar tools from the `CalendarToolset`, enabling it to interact with the Calendar API and respond to user queries regarding calendar states or modifications. The agent's instructions dynamically incorporate the current date for temporal context. To illustrate how an agent is constructed, let's examine a key section from the calendar_agent found in the A2A samples on GitHub.

The code below shows how the agent is defined with its specific instructions and tools. Please note that only the code required to explain this functionality is shown; you can access the complete file [here](#):

https://github.com/a2aproject/a2a-samples/blob/main/samples/python/agents/birthday_planner_adk/calendar_agent/_main_.py

```
def main(host: str, port: int):
    # Verify an API key is set.
    # Not required if using Vertex AI APIs.
    if os.getenv('GOOGLE_GENAI_USE_VERTEXAI') != 'TRUE' and not
os.getenv(
    'GOOGLE_API_KEY'
):
    raise ValueError(
        'GOOGLE_API_KEY environment variable not set and '
        'GOOGLE_GENAI_USE_VERTEXAI is not TRUE.'
    )

    skill = AgentSkill(
        id='check_availability',
        name='Check Availability',
        description="Checks a user's availability for a time using
their Google Calendar",
        tags=['calendar'],
        examples=['Am I free from 10am to 11am tomorrow?'],
    )

    agent_card = AgentCard(
        name='Calendar Agent',
        description="An agent that can manage a user's calendar",
        url=f'http://{host}:{port}/',
        version='1.0.0',
        defaultInputModes=['text'],
        defaultOutputModes=['text'],
        capabilities=AgentCapabilities(streaming=True),
        skills=[skill],
    )

    adk_agent = asyncio.run(create_agent(
        client_id=os.getenv('GOOGLE_CLIENT_ID'),
        client_secret=os.getenv('GOOGLE_CLIENT_SECRET'),
    ))

    runner = Runner(
        app_name=agent_card.name,
        agent=adk_agent,
        artifact_service=InMemoryArtifactService(),
        session_service=InMemorySessionService(),
        memory_service=InMemoryMemoryService(),
    )
```



```

agent_executor = ADKAgentExecutor(runner, agent_card)

async def handle_auth(request: Request) -> PlainTextResponse:
    await agent_executor.on_auth_callback(
        str(request.query_params.get('state')), str(request.url)
    )
    return PlainTextResponse('Authentication successful.')

request_handler = DefaultRequestHandler(
    agent_executor=agent_executor, task_store=InMemoryTaskStore()
)

a2a_app = A2AStarletteApplication(
    agent_card=agent_card, http_handler=request_handler
)
routes = a2a_app.routes()
routes.append(
    Route(
        path='/authenticate',
        methods=['GET'],
        endpoint=handle_auth,
    )
)
app = Starlette(routes=routes)

uvicorn.run(app, host=host, port=port)

if __name__ == '__main__':
    main()

```

This Python code demonstrates setting up an A2A-compliant "Calendar Agent" for checking user availability using Google Calendar. It involves verifying API keys or Vertex AI configurations for authentication purposes. The agent's capabilities, including the "check_availability" skill, are defined within an AgentCard, which also specifies the agent's network address. Subsequently, an ADK agent is created, configured with in-memory services for managing artifacts, sessions, and memory. The code then initializes a Starlette web application, incorporates an authentication callback and the A2A protocol handler, and executes it using Uvicorn to expose the agent via HTTP.

These examples illustrate the process of building an A2A-compliant agent, from defining its capabilities to running it as a web service. By utilizing Agent Cards and ADK, developers can create interoperable AI agents capable of integrating with tools

like Google Calendar. This practical approach demonstrates the application of A2A in establishing a multi-agent ecosystem.

Further exploration of A2A is recommended through the code demonstration at <https://www.trickle.so/blog/how-to-build-google-a2a-project>. Resources available at this link include sample A2A clients and servers in Python and JavaScript, multi-agent web applications, command-line interfaces, and example implementations for various agent frameworks.

At a Glance

What: Individual AI agents, especially those built on different frameworks, often struggle with complex, multi-faceted problems on their own. The primary challenge is the lack of a common language or protocol that allows them to communicate and collaborate effectively. This isolation prevents the creation of sophisticated systems where multiple specialized agents can combine their unique skills to solve larger tasks. Without a standardized approach, integrating these disparate agents is costly, time-consuming, and hinders the development of more powerful, cohesive AI solutions.

Why: The Inter-Agent Communication (A2A) protocol provides an open, standardized solution for this problem. It is an HTTP-based protocol that enables interoperability, allowing distinct AI agents to coordinate, delegate tasks, and share information seamlessly, regardless of their underlying technology. A core component is the **Agent Card**, a digital identity file that describes an agent's capabilities, skills, and communication endpoints, facilitating discovery and interaction. A2A defines various interaction mechanisms, including synchronous and asynchronous communication, to support diverse use cases. By creating a universal standard for agent collaboration, A2A fosters a modular and scalable ecosystem for building complex, multi-agent Agentic systems.

Rule of thumb: Use this pattern when you need to orchestrate collaboration between two or more AI agents, especially if they are built using different frameworks (e.g., Google ADK, LangGraph, CrewAI). It is ideal for building complex, modular applications where specialized agents handle specific parts of a workflow, such as delegating data analysis to one agent and report generation to another. This pattern is also essential when an agent needs to dynamically discover and consume the capabilities of other agents to complete a task.

Visual summary

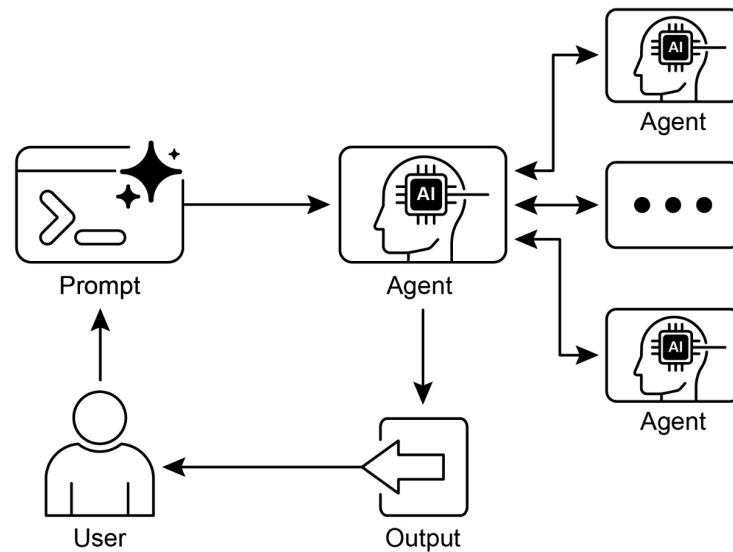


Fig.2: A2A inter-agent communication pattern

Key Takeaways

Key Takeaways:

- The Google A2A protocol is an open, HTTP-based standard that facilitates communication and collaboration between AI agents built with different frameworks.
- An AgentCard serves as a digital identifier for an agent, allowing for automatic discovery and understanding of its capabilities by other agents.
- A2A offers both synchronous request-response interactions (using `tasks/send`) and streaming updates (using `tasks/sendSubscribe`) to accommodate varying communication needs.
- The protocol supports multi-turn conversations, including an `input-required`

state, which allows agents to request additional information and maintain context during interactions.

- A2A encourages a modular architecture where specialized agents can operate independently on different ports, enabling system scalability and distribution.
- Tools such as Trickle AI aid in visualizing and tracking A2A communications, which helps developers monitor, debug, and optimize multi-agent systems.
- While A2A is a high-level protocol for managing tasks and workflows between different agents, the Model Context Protocol (MCP) provides a standardized interface for LLMs to interface with external resources

Conclusions

The Inter-Agent Communication (A2A) protocol establishes a vital, open standard to overcome the inherent isolation of individual AI agents. By providing a common HTTP-based framework, it ensures seamless collaboration and interoperability between agents built on different platforms, such as Google ADK, LangGraph, or CrewAI. A core component is the Agent Card, which serves as a digital identity, clearly defining an agent's capabilities and enabling dynamic discovery by other agents. The protocol's flexibility supports various interaction patterns, including synchronous requests, asynchronous polling, and real-time streaming, catering to a wide range of application needs.

This enables the creation of modular and scalable architectures where specialized agents can be combined to orchestrate complex automated workflows. Security is a fundamental aspect, with built-in mechanisms like mTLS and explicit authentication requirements to protect communications. While complementing other standards like MCP, A2A's unique focus is on the high-level coordination and task delegation between agents. The strong backing from major technology companies and the availability of practical implementations highlight its growing importance. This protocol paves the way for developers to build more sophisticated, distributed, and intelligent multi-agent systems. Ultimately, A2A is a foundational pillar for fostering an innovative and interoperable ecosystem of collaborative AI.

References

1. Chen, B. (2025, April 22). *How to Build Your First Google A2A Project: A Step-by-Step Tutorial*. Trickle.so Blog.
<https://www.trickle.so/blog/how-to-build-google-a2a-project>
2. Google A2A GitHub Repository. <https://github.com/google-a2a/A2A>

3. Google Agent Development Kit (ADK) <https://google.github.io/adk-docs/>
4. Getting Started with Agent-to-Agent (A2A) Protocol:
<https://codelabs.developers.google.com/intro-a2a-purchasing-concierge#0>
5. Google AgentDiscovery - <https://a2a-protocol.org/latest/>
6. Communication between different AI frameworks such as LangGraph, CrewAI, and Google ADK <https://www.trickle.so/blog/how-to-build-google-a2a-project>
7. Designing Collaborative Multi-Agent Systems with the A2A Protocol
<https://www.oreilly.com/radar/designing-collaborative-multi-agent-systems-with-the-a2a-protocol/>

Chapter 16: Resource-Aware Optimization

Resource-Aware Optimization enables intelligent agents to dynamically monitor and manage computational, temporal, and financial resources during operation. This differs from simple planning, which primarily focuses on action sequencing. Resource-Aware Optimization requires agents to make decisions regarding action execution to achieve goals within specified resource budgets or to optimize efficiency. This involves choosing between more accurate but expensive models and faster, lower-cost ones, or deciding whether to allocate additional compute for a more refined response versus returning a quicker, less detailed answer.

For example, consider an agent tasked with analyzing a large dataset for a financial analyst. If the analyst needs a preliminary report immediately, the agent might use a faster, more affordable model to quickly summarize key trends. However, if the analyst requires a highly accurate forecast for a critical investment decision and has a larger budget and more time, the agent would allocate more resources to utilize a powerful, slower, but more precise predictive model. A key strategy in this category is the fallback mechanism, which acts as a safeguard when a preferred model is unavailable due to being overloaded or throttled. To ensure graceful degradation, the system automatically switches to a default or more affordable model, maintaining service continuity instead of failing completely.

Practical Applications & Use Cases

Practical use cases include:

- **Cost-Optimized LLM Usage:** An agent deciding whether to use a large, expensive LLM for complex tasks or a smaller, more affordable one for simpler queries, based on a budget constraint.
- **Latency-Sensitive Operations:** In real-time systems, an agent chooses a faster but potentially less comprehensive reasoning path to ensure a timely response.
- **Energy Efficiency:** For agents deployed on edge devices or with limited power, optimizing their processing to conserve battery life.
- **Fallback for service reliability:** An agent automatically switches to a backup model when the primary choice is unavailable, ensuring service continuity and graceful degradation.

- **Data Usage Management:** An agent opting for summarized data retrieval instead of full dataset downloads to save bandwidth or storage.
- **Adaptive Task Allocation:** In multi-agent systems, agents self-assign tasks based on their current computational load or available time.

Hands-On Code Example

An intelligent system for answering user questions can assess the difficulty of each question. For simple queries, it utilizes a cost-effective language model such as Gemini Flash. For complex inquiries, a more powerful, but expensive, language model (like Gemini Pro) is considered. The decision to use the more powerful model also depends on resource availability, specifically budget and time constraints. This system dynamically selects appropriate models.

For example, consider a travel planner built with a hierarchical agent. The high-level planning, which involves understanding a user's complex request, breaking it down into a multi-step itinerary, and making logical decisions, would be managed by a sophisticated and more powerful LLM like Gemini Pro. This is the "planner" agent that requires a deep understanding of context and the ability to reason.

However, once the plan is established, the individual tasks within that plan, such as looking up flight prices, checking hotel availability, or finding restaurant reviews, are essentially simple, repetitive web queries. These "tool function calls" can be executed by a faster and more affordable model like Gemini Flash. It is easier to visualize why the affordable model can be used for these straightforward web searches, while the intricate planning phase requires the greater intelligence of the more advanced model to ensure a coherent and logical travel plan.

Google's ADK supports this approach through its multi-agent architecture, which allows for modular and scalable applications. Different agents can handle specialized tasks. Model flexibility enables the direct use of various Gemini models, including both Gemini Pro and Gemini Flash, or integration of other models through LiteLLM. The ADK's orchestration capabilities support dynamic, LLM-driven routing for adaptive behavior. Built-in evaluation features allow systematic assessment of agent performance, which can be used for system refinement (see the Chapter on Evaluation and Monitoring).

Next, two agents with identical setup but utilizing different models and costs will be defined.

```

# Conceptual Python-like structure, not runnable code

from google.adk.agents import Agent
# from google.adk.models.lite_llm import LiteLlm # If using models
not directly supported by ADK's default Agent

# Agent using the more expensive Gemini Pro 2.5
gemini_pro_agent = Agent(
    name="GeminiProAgent",
    model="gemini-2.5-pro", # Placeholder for actual model name if
different
    description="A highly capable agent for complex queries.",
    instruction="You are an expert assistant for complex
problem-solving."
)

# Agent using the less expensive Gemini Flash 2.5
gemini_flash_agent = Agent(
    name="GeminiFlashAgent",
    model="gemini-2.5-flash", # Placeholder for actual model name if
different
    description="A fast and efficient agent for simple queries.",
    instruction="You are a quick assistant for straightforward
questions."
)

```

A Router Agent can direct queries based on simple metrics like query length, where shorter queries go to less expensive models and longer queries to more capable models. However, a more sophisticated Router Agent can utilize either LLM or ML models to analyze query nuances and complexity. This LLM router can determine which downstream language model is most suitable. For example, a query requesting a factual recall is routed to a flash model, while a complex query requiring deep analysis is routed to a pro model.

Optimization techniques can further enhance the LLM router's effectiveness. Prompt tuning involves crafting prompts to guide the router LLM for better routing decisions. Fine-tuning the LLM router on a dataset of queries and their optimal model choices improves its accuracy and efficiency. This dynamic routing capability balances response quality with cost-effectiveness.


```

# Conceptual Python-like structure, not runnable code

from google.adk.agents import Agent, BaseAgent
from google.adk.events import Event
from google.adk.agents.invocation_context import InvocationContext
import asyncio

class QueryRouterAgent(BaseAgent):
    name: str = "QueryRouter"
    description: str = "Routes user queries to the appropriate LLM agent based on complexity."

    async def _run_async_impl(self, context: InvocationContext) -> AsyncGenerator[Event, None]:
        user_query = context.current_message.text # Assuming text input
        query_length = len(user_query.split()) # Simple metric: number of words

        if query_length < 20: # Example threshold for simplicity vs. complexity
            print(f"Routing to Gemini Flash Agent for short query (length: {query_length})")
            # In a real ADK setup, you would 'transfer_to_agent' or directly invoke
            # For demonstration, we'll simulate a call and yield its response
            response = await gemini_flash_agent.run_async(context.current_message)
            yield Event(author=self.name, content=f"Flash Agent processed: {response}")
        else:
            print(f"Routing to Gemini Pro Agent for long query (length: {query_length})")
            response = await gemini_pro_agent.run_async(context.current_message)
            yield Event(author=self.name, content=f"Pro Agent processed: {response}")

```

The Critique Agent evaluates responses from language models, providing feedback that serves several functions. For self-correction, it identifies errors or inconsistencies, prompting the answering agent to refine its output for improved

quality. It also systematically assesses responses for performance monitoring, tracking metrics like accuracy and relevance, which are used for optimization.

Additionally, its feedback can signal reinforcement learning or fine-tuning; consistent identification of inadequate Flash model responses, for instance, can refine the router agent's logic. While not directly managing the budget, the Critique Agent contributes to indirect budget management by identifying suboptimal routing choices, such as directing simple queries to a Pro model or complex queries to a Flash model, which leads to poor results. This informs adjustments that improve resource allocation and cost savings.

The Critique Agent can be configured to review either only the generated text from the answering agent or both the original query and the generated text, enabling a comprehensive evaluation of the response's alignment with the initial question.

```
CRITIC_SYSTEM_PROMPT = """
You are the Critic Agent, serving as the quality assurance arm of
our collaborative research assistant system. Your primary function is
to meticulously review and challenge information from the
Researcher Agent, guaranteeing accuracy, completeness, and unbiased
presentation.
Your duties encompass:
* Assessing research findings for factual correctness,
thoroughness, and potential leanings.
* Identifying any missing data or inconsistencies in reasoning.
* Raising critical questions that could refine or expand the
current understanding.
* Offering constructive suggestions for enhancement or exploring
different angles.
* Validating that the final output is comprehensive and balanced.
All criticism must be constructive. Your goal is to fortify the
research, not invalidate it. Structure your feedback clearly, drawing
attention to specific points for revision. Your overarching aim is to
ensure the final research product meets the highest possible quality
standards.
"""
```

The Critic Agent operates based on a predefined system prompt that outlines its role, responsibilities, and feedback approach. A well-designed prompt for this agent must clearly establish its function as an evaluator. It should specify the areas for critical focus and emphasize providing constructive feedback rather than mere dismissal. The